Table of contents

Series 4: Weakest Precondition with Loops and Verification

Methods

Weakest Precondition (WP) Calculus

A backward reasoning method for verifying the partial correctness of a program. We compute the weakest condition on the initial state that guarantees the postcondition is satisfied after program execution.

- **WP for assignment:** $WP(x := e, Q) = Q[x \leftarrow e]$
- **WP for sequence:** $WP(S1; S2, Q) = WP(S1, WP(S2, Q))$
- **WP for `while` loop:** This is the most complex part. For a loop `while B do S done` with an invariant I, we have:

$$WP(\text{while } B \text{ do } S \text{ done}, Q) = I \wedge \forall \vec{v}, ((B \wedge I) \Rightarrow WP(S, I)) \wedge ((\neg B \wedge I) \Rightarrow Q)$$

  where $\vec{v}$ are the variables modified in the loop. In practice, the proof decomposes into three proof obligations:

  1. **Initialization:** The precondition implies the invariant.
  2. **Preservation:** If the invariant and loop condition are true before the body, then the invariant remains true after.
  3. **Postcondition:** If the invariant is true and the loop condition is false, then the postcondition is true.

Finding and Proving Loop Invariants

An invariant I is a property that is true before the first iteration, preserved by each iteration, and thus true upon loop exit.

- **Strategy:** The invariant often relates mutable variables to initial variables (denoted `a0`, `b0`) or expresses a relation that must be maintained (e.g., `a + b = constant`).
- **Verification:** We prove the preservation condition in the form of a Hoare triple: $\{B \wedge I\}S\{I\}$.

Proving Termination with Variants

A variant V is an integer-valued expression (or in a well-founded set) that:

1. **Is bounded below:** Typically $V \geq 0$ when the loop condition B is true.
2. **Decreases strictly:** Its value decreases with each iteration.

This guarantees a finite number of iterations.

- **Strategy:** Choose an expression that clearly measures the remaining "steps" (e.g., a decrementing variable).

Formal Verification with Why3

Why3 is a tool for deductive verification of programs.

- **Encoding:** Translate code, specifications (pre/postconditions), invariants, and variants into the Why3 language.
- **Proof obligation generation:** Why3 automatically generates logical formulas corresponding to partial correctness and termination conditions.
- **Proof:** These obligations can be discharged automatically by solvers (SMT) or manually by interactive provers.

Reminder

- **Hoare logic:** $\{P\}C\{Q\}$ means: if P is true before executing C, and if C terminates, then Q is true after.
- **Rule for `while` loop:**

$$\frac{\{I \wedge B\}S\{I\}}{\{I\} \text{ while } B \text{ do } S \text{ done } \{\neg B \wedge I\}}$$

- **Termination:** A variant V ensures termination if $\{I \wedge B \wedge V = K\}S\{V < K\}$ for some K (and $V \geq 0$ under $I \wedge B$).
- **Assignment rule:** To prove $\{P\}x := e\{Q\}$, it suffices that $P \Rightarrow Q[x \leftarrow e]$.

Illustrative Examples

Example 1: WP Calculation and Invariant for `stupidadd`

Context:

Function `stupidadd(a, b)` that adds `b` to `a` via increments and decrements.

```
stupidadd(a,b) = while (b > 0) do a := a + 1; b := b - 1 done; return a
```

Specification: $\{b \geq 0\}$ stupidadd $\{res = a_0 + b_0\}$.

Application:

1. **Choice of invariant `I1`:** $I_1 \equiv (b \geq 0) \wedge (a + b = a_0 + b_0)$.
   - It expresses that the total sum remains constant and `b` is never negative during the loop.
2. **Proof of the invariant:**
   - **Initialization:** Before the loop, `a = a0`, `b = b0`. So `b0 0 b 0` and `a + b = a0 + b0`. The precondition guarantees `b0 0`.
   - **Preservation:** We must show $\{b > 0 \wedge I_1\}a := a + 1; b := b - 1\{I_1\}$. Compute the WP for the loop body relative to `I1`:

     $$WP(a := a + 1; b := b - 1, I_1) = WP(a := a + 1, WP(b := b - 1, I_1))$$

     $WP(b := b - 1, (b \geq 0) \wedge (a + b = a_0 + b_0)) = ((b - 1) \geq 0) \wedge (a + (b - 1) = a_0 + b_0)$

     $WP(a := a+1, (b-1 \geq 0) \wedge (a+b-1 = a_0+b_0)) = (b-1 \geq 0) \wedge ((a+1)+b-1 = a_0+b_0)$

     This simplifies to: $(b \geq 1) \wedge (a + b = a_0 + b_0)$.
     The precondition for the body is $b > 0 \wedge I_1 \equiv (b > 0) \wedge (b \geq 0) \wedge (a + b = a_0 + b_0)$.
     It is evident that $(b > 0) \wedge (a + b = a_0 + b_0) \Rightarrow (b \geq 1) \wedge (a + b = a_0 + b_0)$.
     The preservation condition is therefore verified.
3. **Postcondition:** Upon loop exit, $\neg B \wedge I_1$, i.e., $b \leq 0 \wedge b \geq 0 \wedge a + b = a_0 + b_0$. This implies $b = 0$ and thus $a = a_0 + b_0$. The function returns `a`, so $res = a_0 + b_0$.

Example 2: Termination Variant for `euclidiv`

Context:

Function computing the quotient of the Euclidean division of `a` by `b`.

```
euclidiv(a,b) = quot := 0; rest := a; while (rest >= b) do rest := rest - b; quot := quot +
```

Specification: $\{a \geq 0 \wedge b > 0\}$ euclidiv $\{a = b * quot + rest \wedge 0 \leq rest < b\}$. The invariant is given: $I_2 \equiv (rest \geq 0) \wedge (a = b * quot + rest)$.

Application:

To prove termination of the loop `while (rest >= b) do ...`, we define a **variant**.

- **Choice of variant:** $V = rest$.
  - Under the invariant I_2 and loop condition $rest \geq b$, we have $rest \geq 0$.
- **Proof of decrease:** We must show that after an iteration, `rest` has strictly decreased. The loop body performs `rest := rest - b`. Under the condition $rest \geq b > 0$, we have $rest - b < rest$. Therefore, the value of the variant V strictly decreases with each iteration.
- **Conclusion on termination:** Since `rest` is a natural integer that strictly decreases with each iteration and is bounded below (by 0 according to the invariant), the loop cannot execute indefinitely. The program terminates.

Synthesis

This series highlights the central elements of verifying imperative programs with loops:

1. The **weakest precondition calculus** provides a systematic framework for partial correctness.
2. Discovering a **relevant loop invariant** is the key creative step, capturing the essence of the algorithm.
3. The **termination proof** relies on constructing a **variant**, an integer measure that strictly decreases and is bounded below.
4. The **Why3** tool allows automating the generation and discharging of proofs corresponding to these reasoning steps.

Mastering these concepts is fundamental for rigorously specifying and proving the correctness of sequential programs.